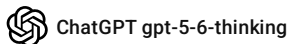


我现在有一个 idea，就是最近在复现那个 Show Harness 那边巨神智能用多模态大模型去驱动巨神智能的一片工作。然后这边工作很好，我在复现的过程中发现一个问题，论文里主要是在用 Gemini、GPT 或者是 Cloud 这种很贵的闭源大模型，能力很强的闭源大模型去上层操控。然后我复现的时候用的是一些比较便宜的小模型，比如说千万 3、VR、9B，或者是 8B，或者是什么这种比较菜的小模型。我觉得在想，这样表现非常非常差。然后我问了我同学和一些专家，他们都表示 Foundation Model，就是基础大模型的能力，在整个闭环当中非常重要。也就是如果你不用 Gemini、Cloud 或者 GPT 的话，用其他的都没用。那我就在想，Agent，这个就是纯数字 Agent，我涉及巨神的时候讨论这个问题，有没有关于这个不同的，就是开源的较弱的小模型，相对于这个闭源的很贵的大模型来说，它的能力到底丢失在哪里，就是它到底差在哪儿？就是有没有关于底层机理的这么一个分析？我们分析出小模型、大模型之间的差距以后，我们就可以针对性地去做一个类似于轻量化的工作。也就是说，比如说一个纯数字 Agent，通过一些其他的一些手段，或者说工程优化方法，或者说架构设计、系统设计、Harness 系统设计，让它做到，就是比如说一个小千万 9B 模型能够达到 GPT 和 Cloud 的百分之九十几的能力，同时成本肯定降了很多，就是这种经济化的、轻量化的，有没有一些思路？当然前提是要分析清楚机理。然后我们调研完这个纯数字 Agent 以后呢，我们就看看有没有一些方法能够把这个轻量化、精简化的一些手段迁移到这个具身，尤其是 Agentic Embodied AI 这个方向上。现在请你仔细理解一下我的意思，然后开始详细的调研和讨论。



我理解你的核心不是“把小模型换个 prompt 试试”，而是先回答一个更基础的问题：为什么同一个 Agent/Harness 闭环里，8B/9B 开源模型会比 GPT/Claude/Gemini 崩得这么厉害；这个差距究竟来自感知、状态维护、规划、工具调用、错误恢复，还是 Harness 对模型能力的隐含假设。然后再做“能力补偿型系统设计”，尽量用便宜模型复现强模型的闭环能力，最后迁移到 Agentic Embodied AI。这个方向是成立的，但“9B 达到闭源模型 90%”不能先当结论，应该把它设计成可验证目标。

目前已经看到一个很关键的反例：你观察到“直接把强闭源 VLM 换成 8B/9B 就垮”，这没问题；但不能进一步推出“小模型做 Agent 没用”。2026 年已经出现专门训练的 9B computer-use agent，在特定浏览器任务上做到很强；Show-Harness 自己也明确报告“小型开源 VLM 经少量适配后可以低成本部署”。所以真正值得研究的不是“模型大小”，而是“通用 foundation capability 中哪些能力对闭环是必要的，以及哪些可以从模型内部搬到 Harness 外部”。

还有一个更重要的查重结果：到 2026 年 9 月，“小模型 + 更好的 Harness”本身已经是明显拥挤赛道。Meta-Harness 做自动 Harness 搜索；HarnessForge 和 Co-Evolving Harnesses and Models 已经直接研究 4B/8B 弱模型与 Harness 的联合适配；最新工作甚至发现，把强模型完整轨迹直接蒸馏给弱模型可能反而掉 4-30 个点，因为会破坏弱模型原来的 planning style 与 Harness 的匹配。

所以如果我们要把这个做成一篇真正有辨识度的工作，重点不能停在“优化 Harness 让 9B 变强”。更有价值的主问题应该是：**强模型到底在闭环的哪些决策节点提供了不可替代的能力，这些能力怎样随 horizon 被放大成最终成功率差距；哪些能力可以外置为确定性/可验证的 Harness 机制，哪些必须留在模型内部。**这才是你这个观察最有可能长成论文的地方。

我把你这个想法和截至 2026 年 9 月 28 日最近的 Agent、Harness、SLM、computer-use、embodied-agent 工作对了一遍。结论是：这个方向有研究价值，但需要把问题重新卡得非常准。简单说成“用更好的 Harness 让 9B 达到 GPT/Claude/Gemini 90%”已经比较拥挤；真正值得做的是研究：

Frontier foundation model 在 Agent 闭环里究竟贡献了哪些不可替代的能力？这种优势为什么会在长闭环中被放大？其中哪些能力可以从模型内部“外置”到 Harness、显式状态、验证器和控制逻辑中，从而让弱模型以显著更低成本接近强模型的系统级能力？

我认为这个版本比单纯“小模型轻量化 Agent”强得多，而且和你正在复现 Show-Harness 产生的真实实验现象能够直接接起来。

一、先纠正一个非常重要的点：你现在对 Show-Harness 的实验，其实发现了一个论文没有真正回答的问题

Show-Harness 原论文并不是在说：

Gemini → 换成 Qwen 2B/9B → 原封不动 zero-shot 也一样好。

实际上它有两种非常不同的 regime。

第一种是 frontier zero-shot：强 VLM 直接承担 perception + reasoning + decision，通过 semantic action units 控机器人。第二种小模型路线则是经过 GUMI demonstrations 微调的 open VLM，而且 fine-tuned mode 甚至把 planner/controller stack 基本拿掉了：每一步输入当前图像和固定 prompt，只预测一个 MV_LEFT / GRASP / DONE 之类的 action token。官方文档明确写的是“no subgoal planner and no stage machinery”。(arXiv) 所以：

你直接把 Gemini/GPT/Claude 替换成 vanilla Qwen 8B/9B，然后发现整个 Agent 闭环崩掉，并不与 Show-Harness 矛盾。

恰恰相反，这暴露了一个非常好的问题：

为什么 frontier VLM 可以作为 general-purpose zero-shot agent brain，而小 VLM 必须经过任务轨迹微调后退化成近似 policy 才能工作？

Show-Harness 证明了“小模型可以学会动作”，但没有真正回答：

小模型为什么无法像 frontier VLM 那样自主闭环思考？

这就是你的入口。

二、目前证据已经非常清楚：小模型与 frontier agent 的差距并不是均匀分布的

我认为这是整个调研里最关键的发现。

看 Berkeley Function Calling Leaderboard V4。

Claude Opus 4.5 和 Qwen3-8B 在最简单的 single-turn function calling 上：

能力	Claude Opus 4.5	Qwen3-8B
Single-turn tool calling	88.58	87.58
Agentic Web Search	84.5	12.0
Agentic Memory	73.76	14.62
Multi-turn	68.38	41.75

也就是说：

Qwen3-8B 并不是连工具都不会调用。

在单步、局部、schema 明确的 action generation 上，它几乎和 Claude 一样。

真正崩掉的是：

需要跨步骤保持状态、搜索、记忆、根据结果重新决策的 **agentic workload**。（Gorilla）这和你 Show-Harness 的观察非常一致。

所以我会把“大小模型差异”初步分成：

$$\text{Agent Gap} \neq \text{Primitive Action Gap}$$

更接近：

$$\text{Agent Gap} = \text{State} + \text{Planning} + \text{Credit Assignment} + \text{Recovery} + \text{Verification} + \text{Long-Horizon Error Accumulation}$$

而不是单纯：

$$\text{Big Model knows more}$$

这会成为论文非常重要的第一个 insight。

三、为什么在闭环里，模型能力差一点最后会差得非常大？

这是你所谓“机理”里面我认为最值得深入研究的部分。

假设一个 Agent 完成任务需要 20 个关键决策。

强模型每个关键步骤正确率 97%。

弱模型每步 90%。

单看：

97% vs 90%。

感觉没有差很多。

但粗略地假设这些步骤必须连续成功：

$$P_{\text{success}} \approx \prod_{t=1}^T p_t$$

20 步之后：

$$0.97^{20} \approx 0.54$$

而

$$0.90^{20} \approx 0.12$$

所以最终任务成功率可以从：

54% → 12%。

这是一种典型的：

closed-loop error amplification

但实际 Agent 比这个更严重，因为错误并不是独立的。

例如第 4 步错误识别当前状态：

$$s_4 \rightarrow \hat{s}_4$$

那么之后：

$$a_5, a_6, a_7 \dots$$

全部是在错误 belief 上做的。

所以真正过程更像：

$$o_t \rightarrow b_t \rightarrow p_t \rightarrow a_t \rightarrow s_{t+1}$$

其中小模型一旦出现：

$$b_t \neq b_t^*$$

后面的 planning 和 action 都会被污染。

因此 frontier model 的优势，很可能不是“每步都强很多”，而是：

它更少进入错误状态，而且进入错误状态之后更容易恢复。

这就是一个真正可以被验证的科学问题。

OSWorld 2.0 最近的分析其实已经给出了很强的旁证：长程 computer-use agent 失败越来越不是因为“不会点按钮”，而是因为它们会丢失约束、遗漏任务中途出现的信息、在存在不确定性时自行猜测而不是询问用户，并且跳过 verification。即便是最强 frontier agent，在他们新的长程任务上也远没有解决问题。(arXiv) 所以这里可以形成第二个 insight：

Agent scaling 的收益主要可能发生在“trajectory management”，而不是 atomic intelligence。

四、我现在认为可以把 Foundation Model 的 Agent 能力拆成 7 个层次

这个拆分非常适合直接用来做你的第一阶段实验。

层	能力	小模型可能的问题	能否 Harness 外置
C1	Perception / Grounding	看错对象、坐标、GUI element	部分可以
C2	Action / Tool Selection	选错工具/primitive	很大程度可以
C3	Argument Construction	参数、格式、schema 错误	非常容易外置
C4	State Tracking	忘记已完成事项、环境变化	很适合外置
C5	Planning / Branching	不会拆任务、错误 branch	部分可以
C6	Failure Recovery	repetition、不会 rollback	很适合外置
C7	Verification / Termination	以为做完、错误 DONE	很适合外置

TRAJECT-Bench 最近就发现 tool agents 的问题会具体表现为 similar-tool confusion、parameter-blind selection，以及随着 trajectory length 增长明显恶化。(arXiv) PEAR 则有一个非常有趣的发现：

planner 弱，比 executor 弱造成的系统退化明显更严重。

换句话说：

$$\Delta_{\text{weak planner}} > \Delta_{\text{weak executor}}$$

这和我们的 hypothesis 非常匹配。([arXiv](#))

五、所以不要直接做“small model vs GPT accuracy”

我建议做一种更强的实验：

Capability Intervention

假设使用：

$$M_s = \text{Qwen 9B}$$

和

$$M_f = \text{frontier model}$$

先运行完全相同的 Harness。

得到弱模型轨迹：

$$\tau_s = (o_1, z_1, a_1, \dots, o_T, z_T, a_T)$$

失败以后，不是只问：

为什么失败？

而是逐能力做 intervention。

例如：

Experiment A：只替换 planner

其他所有东西仍然由 9B 完成：

$$Planner_s \rightarrow Planner_f$$

观察：

$$SR : 25\% \rightarrow 65\%?$$

Experiment B：只替换 state tracking

$$Memory_s \rightarrow StateTracker_{\text{symbolic}}$$

Experiment C：只替换 verifier

$$Verifier_s \rightarrow Verifier_f$$

Experiment D：只替换 recovery

其他仍然是 9B。

最后你就会得到：

$$\Delta SR(C_i)$$

即：

每种 foundation capability 对最终 Agent 成功率的因果贡献。

这比“分析失败 log”强很多。

六、不过这里也必须查重：单纯 failure attribution 已经有人做了

2026 年 failure attribution 已经非常热。

例如 Causal Agent Replay 会重放 Agent 轨迹，对某一步实施 intervention，然后重新执行后面的 trajectory，看最终 outcome 如何变化。CausalFlow 也在做类似的 step-level intervention 和 repair。([arXiv](#)) 所以我们不能写成：

找 Agent 哪一步错了。

这已经不新。

你的区别应该是：

不是 step attribution，而是 capability attribution。

即我们问：

Which capability is responsible for the frontier-small agent gap?

而不是：

Which step caused this particular failure?

这一层区别非常重要。

七、真正有意思的 hypothesis：Frontier Intelligence 可能是“稀疏需要”的

这是目前我觉得你这个 idea 最值得赌的假设。

不是每一步都需要 GPT/Claude/Gemini。

很多 Agent trajectory：

text		
Step 1	Understand goal	HARD
Step 2	Open browser	EASY
Step 3	Search	EASY
Step 4	Interpret results	HARD
Step 5	Click	EASY
Step 6	Fill form	EASY
Step 7	Unexpected popup	HARD
Step 8	Close popup	EASY
Step 9	Check constraints	HARD
Step 10	Submit	EASY

frontier intelligence 可能只在：

1、4、7、9

真正必要。

那么：

Frontier calls $\ll$ Total calls

这件事情最近已经有人从 routing 角度看到。

AgentRouter 2026 年 9 月的工作报告，在多步 agent trajectory 中，把不同步骤路由给不同规模模型，可以削减约 72% 成本，同时保留约 97.3% 的 frontier-only quality。([ArcXiv](#)) 但是它回答的是：

什么时候调用哪个模型？

而你可以进一步问：

为什么这些步骤需要强模型？

这是机制层面的区别。

八、然后才进入你真正想做的“Lightweight Agent”

一旦找到：

$$\{C_{critical}\}$$

我们不一定继续调用 GPT。
而是尝试把这些能力从模型里搬出来。
我称它：

Capability Offloading

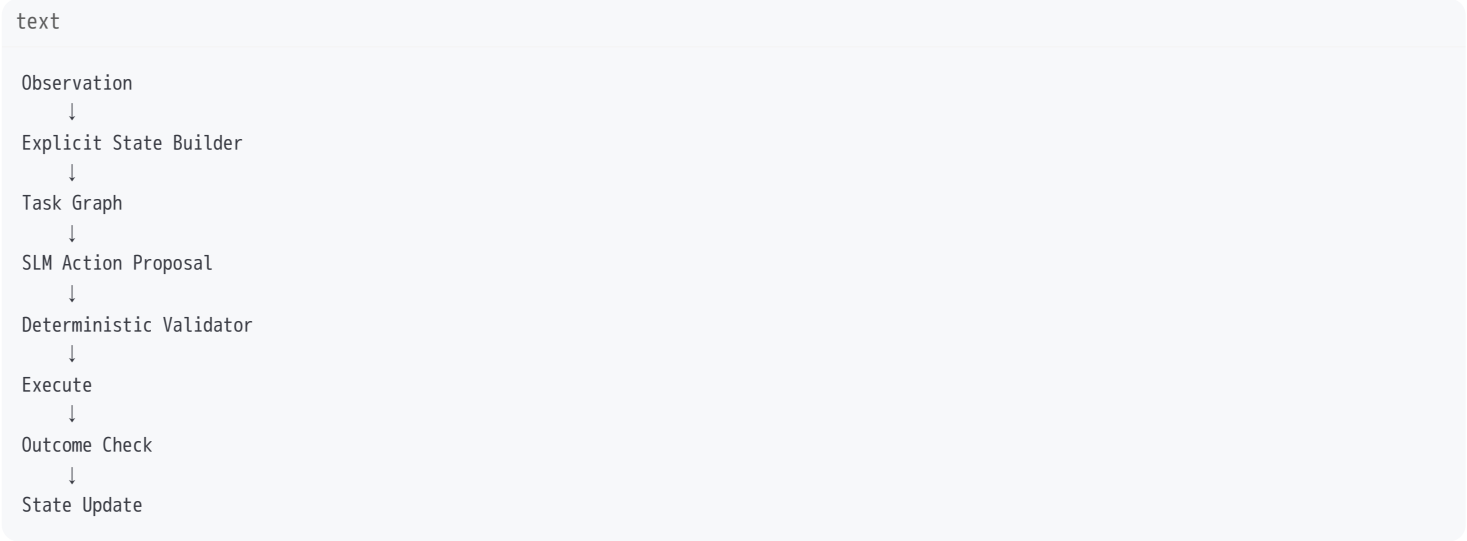
例如原本：

$$LLM : \text{observe} \rightarrow \text{remember} \rightarrow \text{plan} \rightarrow \text{verify} \rightarrow \text{act}$$

改成：

$$\text{Harness State} + \text{Planner} + \text{Verifier} + \text{SLM}$$

具体可以是：



把弱模型最不擅长的东西变成：

explicit state

而不是让模型自己从 20k token history 恢复状态。

例如：

```
json
{
  "goal": "...",
  "completed": [...],
  "pending": [...],
  "constraints": [...],
  "current_state": "...",
  "last_failure": "...",
  "next_valid_actions": [...]
```

那么 SLM 实际要求解的问题就从：

$$P(a_t|o_{1:t}, a_{1:t-1}, g)$$

变成：

$$P(a_t|z_t)$$

其中：

$$z_t = \text{compressed structured state}$$

难度会下降非常多。

九、这里有一个非常漂亮的理论解释

大模型拥有很强的：

implicit computation

它可以在 hidden states / context 里面隐式维护：

- goal
- state
- history
- constraints
- plan
- uncertainty

小模型 capacity 不够，所以隐式同时维护这些变量时开始互相干扰。

我们的 Harness 做的本质是：

Implicit latent computation → Explicit external computation

也就是：

把 intelligence 从参数空间搬到 system structure。

这比单纯 prompt engineering 高一个层次。

可以写成：

C_system = C_model + C_externalized

而目标就是：

C_system(M_small, H*) ≈ C_system(M_frontier, H_0)

同时：

Cost(M_small, H*) ≪ Cost(M_frontier, H_0)

这就是你的 economics story。

十、但“Harness optimization”已经很卷，所以一定不能停在这里

截至现在：

Meta-Harness 已经自动搜索 Harness code，而且在 TerminalBench-2 等任务上能提升固定模型，同时减少 context。(arXiv) HarnessForge 已经联合优化 Harness 和 Qwen3-4B/8B policy，报告相对强 baseline 最高约 12% improvement。(arXiv) Co-Evolving Harnesses and Models 更进一步发现了一个特别重要的现象：强模型完整 trajectory 直接拿来教弱模型会破坏 model-harness fit，在他们七个任务上甚至下降 4–30 points；改成只纠正弱模型自己 rollout 中出错的局部 turn 更有效。(arXiv) 所以如果我们论文叫：

Optimizing Harnesses for Small Agents

基本没戏。

十一、我认为真正可以形成论文的故事应该是下面这个

我暂时给它一个工作标题：

Where Does Frontier Intelligence Matter?

Diagnosing and Externalizing Foundation-Model Capabilities for Lightweight Agents

核心问题不是模型压缩，而是：

Frontier Agent → Capability Decomposition → Critical Capability Identification → Capability Externalization → Small Agent
整个论文三个阶段。

Stage 1: Gap Mapping

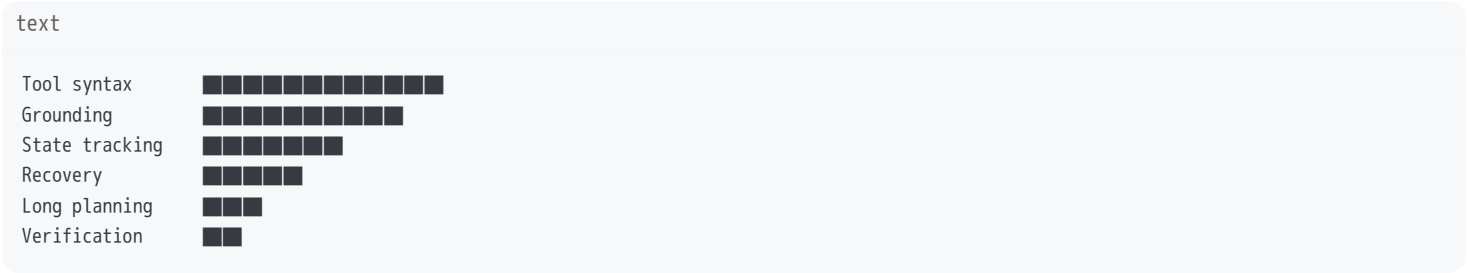
系统测：

2B, 4B, 8/9B, 27/32B, Frontier

随着模型能力下降：

哪个能力首先掉？

可能得到：

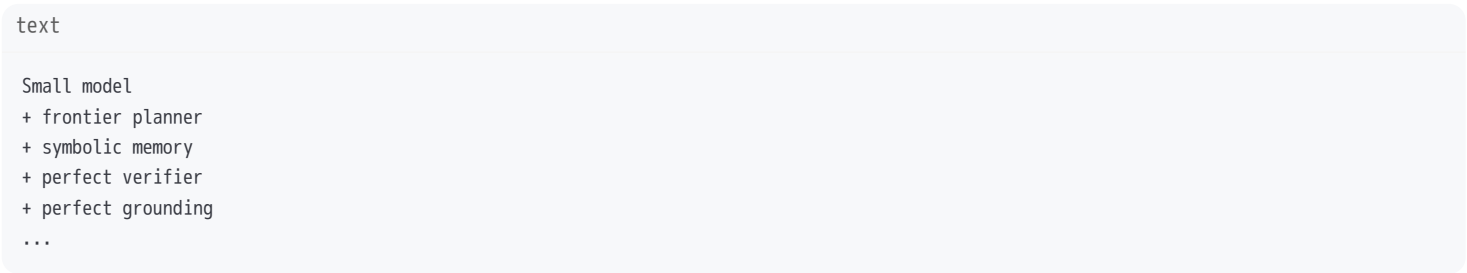


这会形成一个：

Agent Capability Scaling Profile

Stage 2: Capability Intervention

逐个 oracle replacement：



从而得到：

$$ACE_i = SR(do(C_i = C_i^*)) - SR$$

这里的 ACE 就是 capability contribution。

最终很可能会发现：

agent gap 80% 可能来自两三个 capability bottleneck，而不是所有能力。

如果真测出来，这个结果本身就很有价值。

Stage 3: Capability Compilation / Offloading

再针对那几个瓶颈，设计最轻量的 replacement。

例如如果 state tracking 最重要：

structured state ledger。

如果 verification 最重要：

execution-grounded verifier。

如果 planning 最重要：

explicit task graph + constrained planner。

如果 recovery 最重要：

failure state machine。

最终比较：

Frontier

VS

Vanilla 9B

VS

9B + generic agent scaffold

VS

9B + our capability – aware scaffold

最终画：

Success Rate vs Cost

Pareto frontier。

而不是只报 SR。

十二、这里有一个已经证明“你这个大方向不是幻想”的非常强案例：Fara1.5-9B

Microsoft 今年的 Fara1.5 非常值得你精读。

它就是一个 9B computer-use agent。

在 Online-Mind2Web：

Fara1.5-9B：

63.4%

WebVoyager：

86.6%

而相同规模旧模型明显更差。更有意思的是，Fara1.5 的训练体系其实正好体现了我们前面说的几个东西：

- trajectory data；
- grounding；
- VQA；
- user interaction；
- explicit context management actions；
- verifier；
- synthetic environments；
- strong frontier solver 生成高质量 trajectories。([arXiv](#)) 它非常清楚地说明：

9B 并不是天生无法成为强 Agent。

而是 vanilla 9B 缺的是：

agent-specific capability shaping

这和你观察并不冲突。

十三、WebWorld 甚至给出了另一个非常强的证据

原始 Qwen3-8B：

WebArena：

9.8%

经过 WebWorld trajectory training：

20.7%

而 Qwen3-14B：

15.1% → 24.3%

这说明小模型的 agent gap 有相当大一部分不是 immutable model capacity，而是可以通过 interaction experience 恢复。[\(GitHub\)](#) 所以最终可能出现三类能力：

$$C = C_{\text{intrinsic}} \cup C_{\text{trainable}} \cup C_{\text{externalizable}}$$

其中：

Intrinsic

必须依赖模型 capacity。

Trainable

少量 agent trajectory training 可以获得。

Externalizable

没必要放模型里，Harness 直接做得更好。

我认为这三个集合非常值得研究。

十四、然后你的 embodied 迁移就自然了

这里千万不要写成：

我们数字 Agent 做完以后顺便去机器人试试。

那样会很散。

应该提出一个统一问题：

Agent intelligence 到底应该放在哪里？

Digital Agent：

$$Model \leftrightarrow Harness \leftrightarrow Tools$$

Embodied Agent：

$$VLM \leftrightarrow Embodied\ Harness \leftrightarrow Robot$$

结构其实完全一样。

Show-Harness 已经有：

- subgoal planning
- situated planning
- action chunking
- adaptive step
- affordance
- action history
- recovery

这些 plugin 本质就是在做：

capability externalization。[\(GitHub\)](#) 因此你完全可以把数字 Agent 得到的规律迁移成：

Digital

state ledger

→

Robot
scene/task state ledger

Digital
tool verifier

→

Robot
action feasibility verifier

Digital
completion verifier

→

Robot
visual postcondition verifier

Digital
error recovery state machine

→

Robot
grasp / contact / motion recovery

这样就是同一个 scientific hypothesis 在两个世界里验证。

十五、更关键的是：具身这边这几天刚出现一个非常接近的工作

9 月 24 日刚出来的 World Action Agent 需要认真盯。

它让 VLM 在一个 visual action workspace 里进行 action rehearsal、preview、revision 和 in-view correction，还用 interaction traces 去训练小 VLM。

它报告：

$$Qwen3.5 - 9B : 1.7\% \rightarrow 43.3\%$$

的 out-of-domain success 提升。(arXiv) 所以“用 Harness trace 教小 VLM 控机器人”已经不能作为你的主要 novelty。

但这恰恰进一步说明我们要把故事往前推进：

WAA / Show-Harness 告诉我们“可以做到”；
我们研究“为什么能做到、能力到底从哪里来、哪些必须学、哪些应该外置”。

这个区别就出来了。

十六、我现在最看好的核心 insight

如果最后实验支持，我希望论文能得到这样一个结论：

Frontier models do not win agentic tasks because every action requires frontier intelligence. They win because a small number of trajectory-critical decisions require strong state abstraction, branching, recovery, and verification; errors at these points propagate through the closed loop. By identifying and externalizing these bottleneck capabilities, much of frontier-level agentic behavior can be recovered with substantially smaller models.

翻成中文就是：

强模型的 Agent 优势不是“每一步都聪明很多”，而是它在少数决定轨迹命运的关键节点上更可靠；小模型的局部错误又会被长程闭环放大。只要把这些关键能力识别出来并系统外置，就可能用小模型恢复很大一部分系统能力。

这个故事我认为非常漂亮。

十七、实验上我建议先别碰很大的训练

你现在最适合做的是一个很干净的 Phase 0。

直接在你已有 Show-Harness + 一个 digital agent codebase 上做。

模型固定一个 family：

Qwen3.5-2B/4B/9B

再加：

GPT/Claude/Gemini

然后只做：

- 1. 收集完整 trajectory。
- 2. 建 capability-level failure taxonomy。
- 3. 自动/人工标记第一次 divergence。
- 4. 做 frontier replacement intervention。
- 5. 测哪一种 replacement 恢复最终任务。
- 6. 测 horizon 和 gap 的关系。

先别训练。

这样两周内就能知道这 idea 是真现象还是我们想多了。

特别是这三个实验我认为必须优先：

Planner Swap

State Swap

Verifier Swap

如果出现：

text	
Vanilla 9B	22%
+ Better tool schema	26%
+ Frontier planner	51%
+ Explicit state	58%
+ Verifier	63%
Frontier model	69%

那这个项目马上有戏。

如果所有单项 replacement 都没明显提升，那说明差距是 distributed model capacity，这个方向就需要及时停掉。

十八、所以对于你最开始那个问题，我现在可以比较明确地回答

“弱开源小模型相对 GPT/Claude/Gemini 到底差在哪里？”

目前已有证据并不支持“主要差在基础工具调用能力”。

更有可能集中在：

stateful long-horizon orchestration

包括状态维护、规划分支、约束保持、异常恢复、不确定性处理和结果验证。BFCL 上 8B 模型 single-turn tool use 已经接近 frontier，但 agentic search/memory 出现巨大落差，就是很强的证据。(Gorilla) 而“能不能通过 Harness 让小模型接近大模型？”

答案也是：

可以大幅缩小差距，而且已有很多工作证明这一点；但目前并没有证据说明任意任务上 9B 都能做到 frontier 的 90%+。

Fara1.5、AgentRouter、Meta-Harness 等已经说明在特定 domain，这种 economics 是现实可行的。(arXiv) 所以我们不能把“90%”写成 assumption。

更科学的目标应该是：

How much frontier capability can be recovered, at what cost, by externalizing which capabilities?

这句话我认为就是这个项目目前最好的 research question。

如果按照这个方向推进，我下一步建议直接把它进一步收敛成一份“可投稿论文方案”：我会具体定下 paper claim、3-4 个 research questions、实验矩阵、模型、digital benchmark、Show-Harness 具身实验、每个 ablation、指标以及和 Meta-Harness / HarnessForge / Fara / WAA 的逐篇 novelty 对照。

2026-09-28 13:18:52 You 

“how-Harness 原论文并不是在说：

> Gemini → 换成 Qwen 2B/9B → 原封不动 zero-shot 也一样好。

实际上它有两种非常不同的 regime”这个我当然知道，这正是我们的 motivation 之一，小模型如果只能靠 finetune 是很不好的，你的分析非常到位，整体 insight 也很好。更可贵的是把 foundation model agent 的能力拆成七层，这很好，我们就围绕这个，仔细分析大小开闭源 agent 之间到底差在哪里。关键是我们不计划做数字 agent，我们只关注 agentic embodied ai，你前面 cue 到了很多纯数字 agent 的相关工作，我觉得非常好，要总结分析他们的实现思路，从中提取灵感，有没有能迁移（高质量迁移而不是无脑迁移）到具身这边的，回答不要随意换行，乱用回车，稍微紧凑一些。你的分析有点散乱，没有逻辑递进的过程，导致我看不懂，下一次回答注意一下

 ChatGPT gpt-5-6-thinking

我会把问题收紧成一条主线：**不做数字 Agent 本身，只把数字 Agent 当作“机制来源库”，目标是解释 agentic embodied AI 中为什么 frontier VLM 能直接闭环，而小模型在 zero-shot 下崩溃，以及哪些能力能够被具身 Harness 有原则地外置。**接下来我会按“现象→能力分解→机制诊断→数字 Agent 可迁移机制→具身化设计→可验证实验”这条顺序展开，避免再横向堆论文。

进一步查下来，我认为还要先纠正一个实验设计上的混淆：“小/大”和“开源/闭源”不能绑成同一个自变量。GPT/Claude/Gemini 对 Qwen 8B/9B 的差距同时混进了模型容量、预训练规模、多模态 grounding、agent/tool post-training、推理预算以及和原生 Harness 的适配程度。我们真正要解释的是“frontier-agent capability gap”，然后再判断其中多少是规模导致、多少是训练导致、多少其实可以由 Harness 补偿；否则最后很容易得到一个无法解释的“闭源就是强”的结论。

这次我把问题收成一条严格的逻辑链。你现在真正要研究的不是“数字 Agent 如何轻量化”，而是：**在 Agentic Embodied AI 中，frontier VLM 为什么能在 Show-Harness 这种闭环里 zero-shot 工作，而 8B/9B 小 VLM 原封不动接进去就严重退化；这种差距究竟由哪些能力产生，其中哪些能力必须存在于 foundation model 内部，哪些可以通过具身 Harness 显式补偿。**数字 Agent 只作为机制研究的参考系，不作为我们的研究对象。这个定位我认为比上一版清楚得多。

一、首先必须把“大模型 vs 小模型”的问题重新定义，否则实验一开始就会混乱

你现在的真实观察是：

$$\text{Frontier closed VLM} + \text{Show-Harness} \gg \text{small open VLM} + \text{Show-Harness}$$

这个观察成立，但不能立刻解释成“规模越大 Agent 能力越强”。这里至少混了六个因素：

$$\text{Gap} = \text{capacity} + \text{pretraining} + \text{multimodal grounding} + \text{agent post-training} + \text{inference compute} + \text{harness compatibility}$$

比如 Fara1.5 很能说明这一点。同样是 Qwen3.5 backbone，经过专门 computer-use trajectory SFT 后，4B/9B 模型已经能做到很强的 Agent performance；Fara1.5-9B 在 Online-Mind2Web 达到 63.4%，WebVoyager 达到 86.6%。它说明“9B”本身并不等价于“不会 Agent”，大量差距来自 agent-specific training 和系统设计。(GitHub) Show-Harness 恰好提供了另一个极端：frontier VLM 可以 zero-shot 使用语义 action units，而小模型需要 fine-tuning 后才成为有效 policy。(arXiv) 所以我们的研究问题应该正式写成：

What capabilities enable frontier VLMs to operate embodied harnesses zero-shot?

第二问才是：

Which of these capabilities can be externalized to recover small-VLM performance without task-specific fine-tuning?

这两问有严格递进关系。第一阶段是“解释差距”，第二阶段才是“补偿差距”。

二、然后才是我们之前说的七层能力，但这次把它彻底具身化

我仍然认为七层拆解是这个项目目前最重要的骨架。但不能停留在“perception/planning/memory”几个泛词，而要明确它们在机器人闭环里具体是什么。

层	Embodied capability	Frontier VLM 真正做的事	小模型可能出现的 failure
C1	Visual / Spatial Grounding	从多视角 RGB 中找对象、关系、接触位置、空间方向	找错物体、左右/前后错、视角混淆
C2	Action Grounding	把自然语言意图映射为当前可执行 semantic action	动作类型选错、动作对象错
C3	Local Action Parameterization	决定移动方向、幅度、抓/放时机等局部参数	overshoot、premature grasp、动作粒度错误
C4	Embodied State Tracking	维护“现在世界是什么状态”	忘记已经抓住、门已开、物体已移动
C5	Task / Contingency Planning	根据当前状态决定下一 subgoal，并处理分支	rigid plan、不根据环境变化重规划
C6	Failure Detection & Recovery	判断刚才失败了没有，以及怎么恢复	空抓继续执行、重复振荡、错误恢复
C7	Verification & Termination	判断局部/最终目标真的完成了没有	premature DONE、成功后继续乱动

这个拆法和 Show-Harness 本身其实高度对应。Show-Harness 已经有 Multi-View Guidance、Proprioception、Subtask Planning、Situating Planning、Action Chunking、Adaptive Step、Visual Prompt、Action History、Failure Recovery 等插件。[\(GitHub\)](#) 但这里有一个极重要的区别：

Show-Harness 的插件是在问“哪些技巧提高总体性能”；我们要问的是“哪些 capability 是 frontier-small gap 的来源”。

两者不是一回事。

Show-Harness ablation：

$$SR(H) - SR(H - \text{plugin})$$

我们的实验应该测：

$$Gap_i = SR(M_f, C_i) - SR(M_s, C_i)$$

以及：

$$Recovery_i = SR(M_s + \text{oracle } C_i) - SR(M_s)$$

这是从“系统 ablation”变成了“能力归因”。

三、现在来看数字 Agent 文献，真正值得迁移的不是具体模块，而是它们发现的“Agent 为什么失败”

这是我认为这次最重要的部分。

数字 Agent 已经给我们提供了一批机制级证据，而且其中有几个与机器人高度同构。

第一类证据：问题往往不是“不会调用工具”，而是无法处理 state dependency

ToolSandbox 专门把普通 function calling 扩展成 stateful interaction，加入隐式状态依赖、信息缺失、状态变化等条件，结果开源模型和 proprietary models 出现明显差距，而且连最强模型在 State Dependency、Insufficient Information 等任务上也困难。[\(ACL Anthology\)](#) 这个结果非常值得迁移。

数字 Agent 中：

$$tool_2$$

是否合法，取决于：

$$state(tool_1)$$

机器人中完全一样：

$$PLACE$$

是否合法，取决于：

- object grasped?
- container open?
- target reachable?

所以数字 Agent 的真正启发不是“给机器人加 memory”。

而是：

机器人 Harness 需要把隐藏在 VLM context 里的 physical state dependencies 显式化。

这比加一个 textual memory 高一个层次。

四、这使我认为 C4——Embodied State Tracking——可能是第一个真正值得下注的 bottleneck

数字 Agent 长程实验越来越显示，维护 canonical state 非常关键。YC-Bench 在数百轮长程任务中发现，scratchpad / persistent state 的使用与成功强相关。(arXiv) 但机器人这里不能简单照搬：

```
text

Memory:
I picked up the cup.
I moved left.
```

Show-Harness 已经有 Action History，这么做基本没有 novelty。(GitHub) 真正具身化应该变成：

$$B_t = \{O_t, R_t, G_t, P_t, U_t\}$$

例如：

```
text

cup_1:
  visible: true
  pose_relation: left_of_plate
  grasp_state: held
  last_confirmed: t=18
  confidence: 0.93

drawer:
  state: open
  opening_verified: true

robot:
  gripper: closed
  contact: detected
```

重点在两个地方。

第一，它是 **world state**，而不是 action history。

第二，它必须通过 perception / proprioception / execution feedback 更新，而不能因为模型说“我抓到了”就写成 held=true 。

也就是：

$$\text{Model belief} \neq \text{canonical physical state}$$

这一点是数字 Agent 的 stateful runtime 思想真正高质量的 embodied migration。

五、第二个非常强的数字 Agent 启发是：Verification 不能继续交给同一个生成模型

Microsoft 在 computer-use verifier 上做出的一个重要结论是：process 和 outcome 必须分开判断，Verifier 的 rubric 设计本身会显著决定性能，而且已有通用 GUI judge 存在很高 false positive。(微软) 这件事情迁到机器人上非常自然。

现在很多 VLM robot agent 实际上是：

然后还是：

same VLM decides whether action succeeded

这是非常危险的闭环。

因为：

产生错误 hypothesis 的模型，很可能继续合理化自己的错误。

例如：

```
text

VLM:
GRASP cup

机器人执行

VLM:
cup is probably grasped
→ MOVE_UP
```

而我们真正需要的是：

$$Proposal \rightarrow Execute \rightarrow Observation \rightarrow Postcondition\ Check \rightarrow Commit$$

例如 GRASP 的 postcondition：

$$gripper_closed \wedge contact \wedge object_motion_consistent$$

满足才：

$$state(object) = held$$

否则：

$$state(object) = not_held$$

然后触发 recovery。

这个东西不是普通“VLM verifier”，而是：

execution-grounded state transition verification

我认为这是一个很重要的具身化升级。

六、第三个启发：frontier intelligence 很可能不是每一步都需要

AgentRouter 最新研究把 multi-step Agent 中不同 step 分给不同能力层级模型，报告在其任务设置中可大幅降低成本，同时保持接近 frontier-only 的总体质量。核心 observation 是：

trajectory 内不同 step 的 reasoning difficulty 差别非常大。(ArcXiv) 这个发现迁移到机器人非常有意思。

机器人 trajectory 可能是：

text	
识别任务	HARD
找到目标	HARD
MOVE_LEFT	EASY
MOVE_LEFT	EASY
MOVE_FORWARD	EASY
是否进入可抓取范围	HARD
GRASP	MEDIUM
判断是否抓成功	HARD
MOVE_UP	EASY
选择下一子目标	HARD

所以我们不应该假设：

$$C_t = C$$

而可能是：

$$C_t \in \{easy, critical\}$$

这就引出了一个我认为非常值得研究的概念：

Embodied Critical Decision Points

即轨迹中少数对最终成功有巨大影响的决策点。

例如：

- 首次确定目标；
- 进入接触阶段；
- grasp；
- 状态发生突变；
- plan branch；
- recovery；
- DONE。

我们的假设就可以升级成：

$$\text{frontier-small gap}$$

主要集中于：

$$t \in T_{\text{critical}}$$

而不是：

$$\forall t$$

如果这个 hypothesis 成立，意义非常大。

七、但这里不要直接照搬 AgentRouter，否则只是“机器人模型路由”

高质量迁移应该再向前一步。

数字 Agent 的方案是：

$$\begin{aligned} easy\ step &\rightarrow small\ model \\ hard\ step &\rightarrow frontier\ model \end{aligned}$$

但我们的最终目标不是一直调用 Gemini。

我们应该把它改成：

$$easy\ step \rightarrow small\ VLM$$

而：

$$critical\ step \rightarrow small\ VLM + \text{additional embodied structure}$$

比如：

```
text

normal mode:
RGB → 9B → semantic action

critical mode:
RGB
+ structured state
```

```
+ depth / geometry
+ candidate actions
+ verifier
→ 9B
```

也就是说我们不是：

adaptive model routing

而是：

adaptive capability allocation

这个转换我认为非常重要。

它保留了 AgentRouter 的核心 insight：

intelligence demand is temporally sparse

但把 frontier inference 换成了：

temporally sparse external computation。

这才真正符合我们的轻量化目标。

八、第四个非常重要的数字 Agent 启发：不要试图让弱模型“像强模型一样思考”

这个是 Co-Evolving Harnesses and Models 最近给出的一个很值得重视的结果。

他们发现，把强模型完整 trajectory 直接 SFT 给弱模型，在 evolved harness 下反而在七个任务上全部退化，幅度 4–30 points。原因是弱模型学到了强模型的 planning style，却没有足够能力可靠执行；最终破坏了原先的 model-harness compatibility。局部 on-policy correction 反而更好。[\(arXiv\)](#) 这件事情对你这个项目非常重要。

它意味着我们应该放弃一种直觉：

Gemini 很强 → 收 Gemini trajectory → 教 Qwen9B 模仿 Gemini → finished。

这恰好可能是错的。

真正值得做的是：

τ_s = small-model native rollout

找到第一次 capability failure：

t^*

然后问：

如果只把 t^* 对应的能力补上，后面弱模型自己的 trajectory 能不能恢复？

这个实验既是 diagnosis，也是后来 Harness 设计的依据。

这比 imitation 更符合我们的研究目标。

九、第五个数字 Agent 思想：Harness 不应该只是 prompt，而应该承担 computation

Meta-Harness 把 Harness 定义为决定“存什么、取什么、给模型呈现什么”的可执行代码，并展示自动优化 Harness 可以减少大量 context，同时提高性能。[\(arXiv\)](#) harnessForge 更进一步，把 agent 看成：

$$Agent = Harness + Policy$$

并明确指出二者之间存在 compatibility，而不是 Harness 随便换哪个模型都有效。[\(arXiv\)](#) 两个工作对我们真正有价值的启发是：

不要把 Embodied Harness 理解成“机器人 prompt 工程”。

Show-Harness 已经迈出了这一步，因为它有 deterministic interpreters 和 plugins。[\(arXiv\)](#) 们可以进一步定义：

$$H = H_P + H_S + H_D + H_V + H_R$$

其中：

H_P : *perception interface*

H_S : *state runtime*

H_D : *decision constraints*

H_V : *verification*

H_R : *recovery*

换句话说：

Harness 是 VLM 外部的一台“小型认知机器”。

这个表述我认为比“scaffold”更准确。

十、但是这里要非常警惕两篇刚刚出现的机器人工作，因为它们直接压到了我们的空间

这几天新出的 Robo-Harness K1 和 World Action Agent 必须认真看。

Robo-Harness K1 的核心就是：

不要求 VLM 自己从 RGB 隐式恢复所有空间信息，而是把 calibrated depth、persistent visual anchors、spatial measurements、grasp hypotheses 变成可查询 perception tools。

它明确指出 direct RGB-only VLM control 很依赖模型能力。📄(arXiv)说明：

$C1$

——perception/spatial grounding externalization——已经有人占了相当大一块。

所以我们不能把论文核心做成：

给小 VLM 加 depth / point / grasp tools。

那基本会撞 K1。

World Action Agent 则做了：

$Action \rightarrow Rehearsal \rightarrow Preview \rightarrow Revision \rightarrow Execution$

同时有 contact view、imagination agent、in-view correction，并且已经证明用 Harness traces fine-tune Qwen3.5-9B 可以把 OOD performance 从 1.7% 提到 43.3%。📄(arXiv)以：

action preview + correction

和

harness trace $\rightarrow$ small VLM fine-tuning

也不能再作为我们的 headline。

这两个最新工作实际上帮助我们排掉了 many 普通 idea。

十一、排掉这些以后，我现在认为最值得我们继续追的并不是七个 capability 全做，而是先找“真正的 bottleneck 组合”

我现在的 hypothesis 是：

$C4 + C6 + C7$

可能比 $C1-C3$ 更值得研究。

也就是：

State Tracking + Recovery + Verification。

原因是 $C1$ perception 已经被 K1 强烈覆盖， $C2/C3$ semantic action grounding 又是 Show-Harness 的核心，而 planning 也已经被 Show-Harness 的 Subtask/Situated Planning 和 WAA 大量覆盖。📄(GitHub)而 $C4/C6/C7$ 可以形成一个非常统一的问题：

Does the agent know what actually happened?

这是我觉得现在特别有意思的一句话。

Frontier model 的优势可能不是：

更会规划。

而是：

它对“世界当前究竟是什么状态、刚才动作到底有没有成功、现在应该继续还是恢复”有更稳定的闭环判断。

于是整个能力链就是：

$$Action \rightarrow Physical\ transition \rightarrow State\ estimation \rightarrow Verification \rightarrow Commit \rightarrow Next\ decision$$

任何一步错：

$$\hat{s}_t \neq s_t$$

后面所有 reasoning 都建立在错误世界上。

这比单纯“long-horizon planning”更具身。

十二、这样，我们就能形成一个非常清楚、而且与数字 Agent 有真正机制联系的研究故事

数字 Agent 给我们的三个机制发现分别是：

State dependency

Verification matters

Critical intelligence is sparse

然后把它们具身化：

Physical state dependency

Execution-grounded verification

Critical physical decision points

最后形成：

State-Aware Lightweight Embodied Agent

它不是“数字 Agent 方法搬到机器人”。

而是：

数字 Agent 暴露出了长程 Agent 的一般计算瓶颈；机器人把这些瓶颈变得更加严重，因为 state 是部分可观测的、action 会改变物理世界、失败具有持续后果。我们因此重新设计适合物理闭环的外部计算机制。

这才叫高质量迁移。

十三、我建议以后判断一个 digital-agent idea 值不值得迁移，用这三个条件

这也可以成为我们后续调研的筛选原则：

条件	含义
Structural Homology	数字和具身系统面对的是同一种计算问题，而不是名字相同
Embodiment Gain	物理执行、部分可观测、接触、几何或时序使问题变得更有意义
Non-trivial Adaptation	迁移后必须重新定义 representation / algorithm，而不是把 prompt 改成 robot prompt

例如：

Memory → robot textual memory

我会直接否掉。

因为 adaptation 太浅。

但：

stateful tool runtime → verified physical state runtime

我认为值得做。

LLM verifier → physical transition verifier

值得做。

model router → adaptive embodied capability allocation

也值得做。

十四、因此我现在建议整个项目按下面这条递进路线推进，而不是先发明方法

第一阶段先回答：

Where does the frontier-small embodied agent gap come from?

在同一个 Show-Harness 上，对 frontier VLM 和 8B/9B VLM 做严格 matched trajectory analysis。

第二阶段回答：

Which capabilities are causally responsible?

做 C1–C7 oracle intervention。

例如：

```
text

9B
9B + oracle grounding
9B + oracle state
9B + oracle planner
9B + oracle recovery
9B + oracle verifier
```

不是同时加。

然后我们得到：

$$Recovery(C_1), ..., Recovery(C_7)$$

第三阶段才看：

Can the dominant capabilities be externalized?

如果最终发现：

```
text

Oracle perception      +7
Oracle planning        +6
Oracle state           +21
Oracle recovery        +14
Oracle verifier        +18
```

那我们的方法就顺理成章地只针对：

$$C4 + C6 + C7$$

而不是凭感觉发明一大堆 plugin。

十五、我现在对这个项目的认识已经比上一轮更收敛了

目前我们不是在做：

Lightweight Show-Harness。

也不是：

Small VLM robot agent。

更不是：

把 digital-agent memory/planner 搬到 robot。

而是在研究一个更基础的问题：

Where should embodied agent intelligence live?

现在的 frontier system 大量 intelligence 在：

$$M_f$$

里面隐式完成。

我们希望先诊断：

$$M_f - M_s$$

究竟体现在哪些闭环能力上。

再把真正重要且可结构化的部分：

$$C_{\text{externalizable}}$$

迁移到：

$$H$$

最终目标：

$$(M_s, H^*) \approx (M_f, H_0)$$

同时：

$$Cost(M_s, H^*) \ll Cost(M_f, H_0)$$

而且最好：

$$H^*$$

不是针对某几个任务 hard-code，而是对应“physical state、verification、recovery”这样跨 embodiment 都成立的基本 Agent 机制。

我现在认为，这就是最值得继续往下挖的版本。下一步最应该做的不是继续脑暴模块，而是把 C1-C7 每一层如何定义、如何构造诊断任务、如何做 oracle replacement、如何区分相邻 capability，以及 Show-Harness 代码里具体应该在哪里插 intervention 全部设计出来。那一步做完，我们就真正拥有一套可以开始跑实验的“大小 VLM Agent 能力解剖协议”。[memcite](#)